



Consistency Maintenance Based on the Mark & Retrace Technique in Groupware Systems *

Ning Gu, Jiangming Yang and Qiwei Zhang
Department of Computing and Information Technology
Fudan University

No.220 Handan Road, Shanghai 200433, P.R.China

{ninggu,yangjiangming,qiweizhang}@fudan.edu.cn

ABSTRACT

Replicated architecture is widely used for concealing network delay. However, consistency maintenance in fully replicated architecture is a major technical challenge. In this paper, we report a Mark & Retrace based method in replicated groupware systems. Compared with the Operation Transformation technique, it does not adjust the operation's position but retraces the document's address space to the state at the time of the operation's generation. Then the operation can be executed directly in this address space. Mark & Retrace method can not only achieve the same goal of consistency maintenance but also provide a better support for *Undo*. This paper provides the proof of the algorithm's correctness of consistency maintenance, in which both the orders of character nodes and marks of each node at all sites are kept consistent. Furthermore, the amortized efficiency can reach $O(\log n)$.

Categories and Subject Descriptors

C.2.4 [Computer-Communication Networks]: Distributed Systems—*Distributed applications*; H.5.3 [Information Interfaces and Presentation]: Group and Organization Interfaces—*Collaborative computing, Synchronous interaction*

General Terms

Algorithms, Design, Human Factors

Keywords

CSCW, Concurrency Control, Consistency, Group editors, Undo, Address Space Transformation, Mark & Retrace

*The work is supported by NSFC under Grant No.60473124, National Grand Fundamental Research 973 Program of China under Grant No.2005CB321905, Shanghai Yi-Wang-Liang-Ku under Grant No.20051020d1sj05-A and Shanghai Science and Technology Committee Key Project.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. To copy otherwise, to republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee.

GROUP'05, November 6-9, 2005, Sanibel Island, Florida, USA.

Copyright 2005 ACM 1-59593-223-2/05/0011 ...\$5.00.

1. INTRODUCTION

Group editors are one of the most commonly used groupware applications [2, 7, 15]. It allows multiple users to participate in the editing and modification of shared data objects, such as word, graphics, and media documents, using a computer network. Techniques for building group editors have been widely applied and practiced [2, 3, 7, 12]. Group editors have also been acknowledged as powerful research vehicles in exploring solution to consistency problems [2, 4, 6, 15, 19].

To hide the effects of network delay and give better local response to users, a replicated architecture has been widely adopted [2, 6, 15]. In a replicated architecture, a local user's operations are executed immediately to achieve good responsiveness and are then multicast to remote replicas to keep the document states consistent. Compared with conventional techniques such as locking and serialization, it has no adverse effects on the user interface while the replicated document states are maintained in consistent states which observe the CCI model [6, 15].

OT is a consistency maintenance technique. In the OT method, due to the existence of concurrent operations, a remote operation should include or exclude some operations' effects [14, 15] and find the correct operation position or area at the local site. To maintain the CCI model, in the past decade, OT method has been gradually improved (include extensions of control algorithms [2, 14, 15, 8, 17] and OT functions [5, 9]). In this process, we have realized that an OT algorithm which completely maintains the CCI model will be very intricate.

Mark & Retrace is a new method, which is different and independent of the OT technique. It retraces the document's address space to the state at the time of the operation's generation. In this state, we can directly find the execution position for the operation. This process does not affect the order of characters in current document. It only needs to determine the position of a new operation. It can maintain the CCI model in replicated architecture and is easier to support group *Undo* [19, 20].

This paper is organized as follows. First, the concept of the address space transformation is introduced in Section 1. Then, the Mark & Retrace method is described. The correctness and efficiency of the technique and how it supports *Undo* operation are discussed in Sections 3, 4 and 5, respectively. In Section 6, the Mark & Retrace method is compared with related works. Finally, we conclude this paper and point out future directions.

2. ADDRESS SPACE TRANSFORMATION

State vector based timestamping [2] has been widely used to detect causal relationships among operations. In the timestamping scheme, every site maintains its own state vector. After an operation is generated, it is attached with the generating site's current state vector, and is then multicast to all other sites.

Then, we discuss the operation's expression. Without loss of generality, each character is indivisible and each operation targets only one character. In fact, any complex *Insert* or *Delete* can be divided into a group of indivisible operations. So this assumption does not affect operation's capability. Since an operation targets only one character, a *Delete* can only be represented by a position P . A *Delete* is timestamped by $SV_{O_{del}}$, which indicates that the site's state vector is $SV_{O_{del}}$ when the operation was generated and its intention is to delete the P th effective character counting from the left. And an *Insert* can be represented by an Inserted content C and an Inserting position P . An *Insert* is timestamped by $SV_{O_{ins}}$ denotes that the site's state vector is $SV_{O_{ins}}$ when the operation was generated and its intention is to Insert the content C between the P th and $P+1$ th effective characters counting from the left. (There also exist two special Inserting positions: the left of the most left effective character and the right of the most right character).

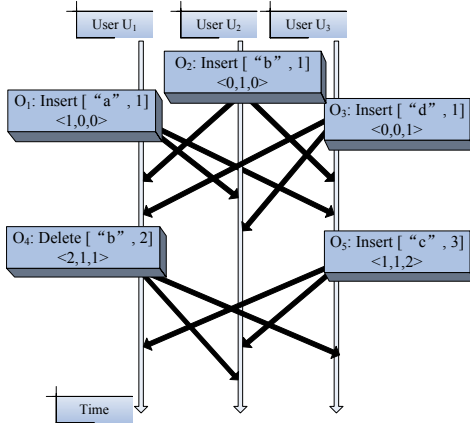


Figure 1: The order of operations' producing and executing

A remote operation cannot be directly executed at the local site due to the effect of concurrent operations on the local document state, which may result in an offset in the execution position of the remote operation. Consider a system consisting of three users and a shared empty document state. The order of operations' generation and execution is shown in Fig.1. Here we only need to focus on operation O_4 and O_5 , but this example is also relevant to later Fig.3 and 4. Operation O_5 is to insert "c" at position 3 between "b" and "d". Consider what happens when user U_1 attempts to execute the operation O_5 —inserting the character "c" at position 3. Since the character "b" was deleted by O_4 , position 3 now refers to the position on the right of "d" rather than the intended position of the left of "d". To resolve this problem, the OT technique transforms the operation's target position. It adjusts the target position of an operation by taking into account of all the effects of concurrent

operations. In Fig.1, due to the existence of concurrent operation O_4 , O_5 shall be transformed to Insert ["c", 2] before execution.

The AST (Address Space Transformation) is a different technique to OT—the operation's intention is achieved by transforming the address space, rather than the operation itself. The basic idea of AST is as follows. If it is known that user U_1 cannot execute O_5 directly because the address space of the document has been changed, address "3" represents the right of character "d" which is different from the one at the time of O_5 's generation. To execute operation O_5 , we shall *retrace* the document's address space to the state at the time of O_5 's generation, which is shown in Fig.2. In this way, O_5 can be executed directly in this new address space, that is address "3" represents the position between "a" and "d". To support the transformation of the document's address space, a marking technique is used. A mark indicates if a character node effective or ineffective at a given time. We shall describe the Mark & Retrace method and how it may be used to achieve consistency in the next section.

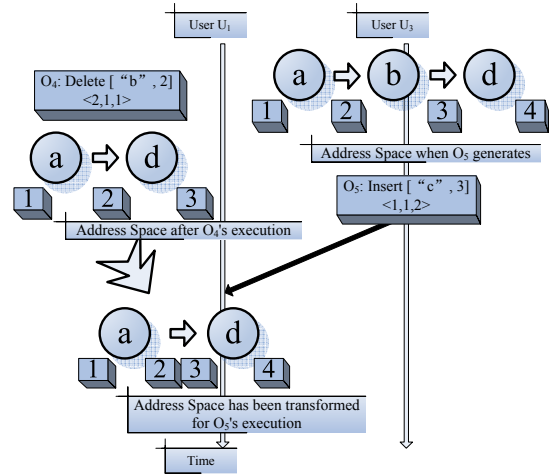


Figure 2: The address space transformation

3. ACHIEVING CONSISTENCY

To achieve causality preservation in the timestamping scheme, we only need to ensure that every remote operation is causally-ready before its execution, which we call execution condition [2, 15]. It should be noted that even if causality is preserved, the execution order of the operations would still be different because of their different orders of arrival at different sites. In the following, we shall focus on the issues of how we can execute an operation correctly to achieve convergence and intention preservation, under the assumption that causality is preserved. In follow discussion we use the same definition of casual ordering relation " $\rightarrow$ " and independent relation " $\parallel$ " in [15].

3.1 The Structure of Mark & Retrace

The difficulty in achieving convergence and intention preservation is that the relationships that exist among independent operations can result in changes to the operations' positions and area. The OT technique searches the operations that may cause these changes and derives their effects first. Then it adjusts the operation current position accordingly

[2, 6, 15]. Mark & Retrace method tries to conceal such effects in the document. It marks and conceals the execution effects of some operations, which allows the document to be converted to the document to the address space at the time of an operation's generation. In this way we can directly find an accurate position and then achieve the execution effects of the operation. The advantage of this method is that we do not have to think of the complex relations among operations. We only need to pay attention to each operation's effect on user's view.

Assume a document is a liner structure composed of characters. Each character may have several operations targeting itself while each operation targets only one character. Then we save the operation together with its timestamp to the character's liner node. So apart from the information of characters and corresponding operations, we also add the effective/ineffective mark information to every node in the liner structure. The mark indicates whether the current node is observable on the user interface or not. So a document structure might be expressed as Fig.3. In Mark & Retrace method, the address space can be expressed as the effective character nodes from left to right.

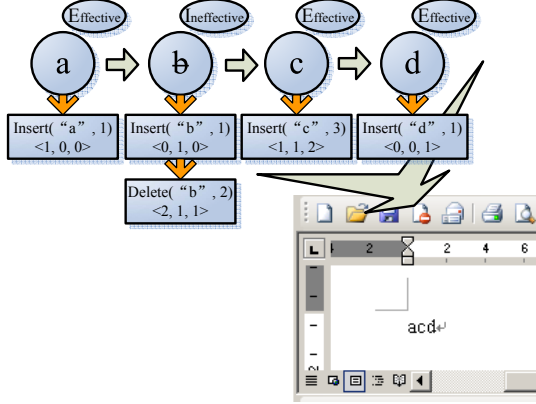


Figure 3: The final state of document's liner structure and user's view in the case of Fig.1

3.2 The Retracing Procedure

Given the liner structure of document at site S , Doc_S , and the state vector at site S , SV_S . Suppose SV is a timestamp which meets the execution condition. Then the Retracing procedure can be expressed as Proc.1:

The goal of the Retracing procedure is to retrace the address space to the state at a given timestamp SV . It reviews every character node and determines their "effective/ineffective" marks in the timestamp SV . Its execution is only to preserve the effects of operations which are causally before generation. For every character node, suppose that the *Insert* and *Delete* attached are timestamped by $SV_{O_{ins}}$ and $SV_{O_{del}}$. We should consider the following four situations. (1) if $SV_{O_{ins}} \geq SV$, then this node is marked ineffective; (2) if $SV_{O_{ins}} \leq SV$ and there is no *Delete* targeting this character node, then this node is marked effective; (3) if $SV_{O_{ins}} \leq SV$ and $SV_{O_{del}} \geq SV$, then this node is marked effective; (4) if $SV_{O_{ins}} \leq SV$ but $SV_{O_{del}} \leq SV$, then this node is marked ineffective.

On one hand, consider an *Insert* and a *Delete* attached on a node, then it must be $SV_{O_{ins}} \leq SV_{O_{del}}$. In other words

Procedure 1 Retracing (Doc_S, SV), retrace the address space of Doc_S with timestamp SV

```

1: for all character node  $CN_i$  of liner structure of  $Doc_S$ 
   do
2:   Set  $CN_i$  ineffective
3:   Consider the Insert  $O_{ins}$  of  $CN_i$ 
4:   if  $O_{ins}$  is timestamped by  $SV_{O_{ins}}$  and  $SV_{O_{ins}} \leq SV$ 
     then
5:     set  $CN_i$  effective
6:   end if
7:   for each Delete  $O_{del}$  of  $CN_i$ 
8:     if  $O_{del}$  is timestamped by  $SV_{O_{del}}$  and  $SV_{O_{del}} \leq SV$ 
       then
9:       set  $CN_i$  ineffective
10:    end if
11:  end for

```

there will not exist the situation where $SV_{O_{ins}} \geq SV$ while $SV_{O_{del}} \leq SV$. On the other hand, when several concurrent *Delete* operations target one character, there may be several *Delete* operations attached on one node. When one of them satisfies $SV_{O_{del}} \leq SV$, this node must be marked ineffective. We can see that inside a node, the effects of operations independent or causally after timestamp SV have not been considered. As to the result of Retracing, an "effective/ineffective" mark of one character node is determinate as long as timestamp SV is fixed.

There are two examples. Given an operation O which is timestamped by SV and is causally-ready for execution. For any operation O' and $O' \rightarrow O$, O' is sure to have been executed. The result of Retracing (Doc_S, SV) is that it can conceal all the effects of operations which are independent of operation O and retrace the address space to the time SV at the time of its generation. In other words, it can achieve an address space of liner structure, whose effective character nodes are completely consistent with the site's liner structure at the time of operation O 's generation. This consistency has two meanings. One is that they both have the same set of effective character nodes. The other is that all the ineffective character nodes at the current site are either ineffective or nonexistent at the operation's generation site and at the time of the operation's generation. Another example. When current state vector of the site S is SV , the result of Retracing(Doc_S, SV) is that it can restore the address space to the state that is used by the local user. If SV represents the state vector when a new operation has just been executed at the site S , the effect of the new operation has already been involved. All the timestamps of operations which have been executed before the new one are all before SV . And the adjustment of SV after new operation's execution also makes new operation's timestamp before SV . So all the effects of executed operations have been retained and the new operation's effect also has been applied.

3.3 The Control Algorithm

When a remote operation O generates at site R and timestamped by SV_O is causally-ready for execution, we can express the execution process of O at site S whose state vector is SV_S in Proc.2.

The definition of operation position is obtained by counting the number of effective character nodes on the left side of linear structure at the time of its generation. When the

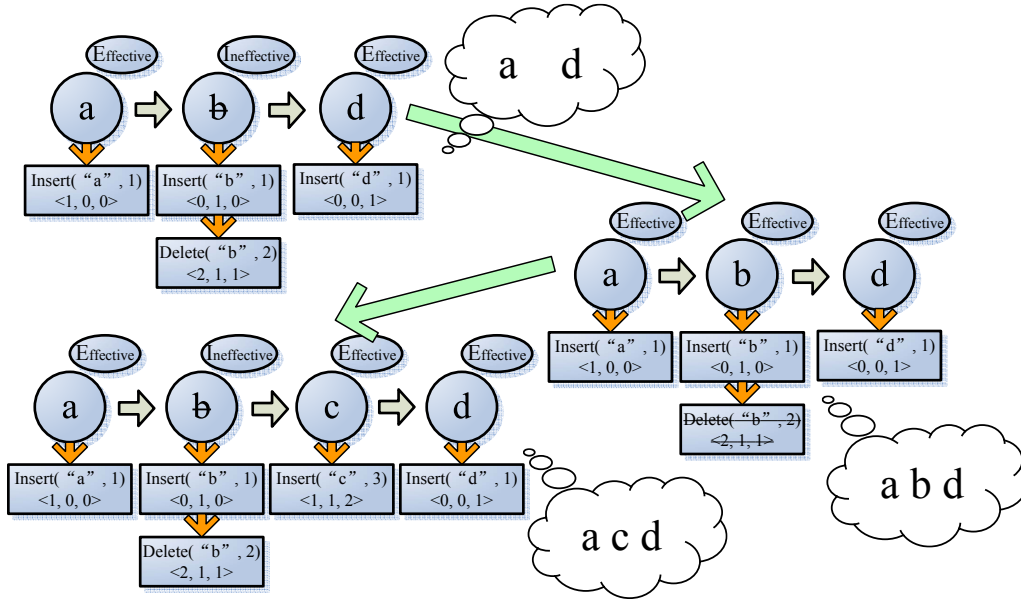


Figure 4: The process of executing operation Insert[“c”, 3] in the case of Fig.1

Procedure 2 Control-Algorithm ($Docs, O$), execute O on $Docs$

- 1: Retracing($Docs, SV_O$)
- 2: Execute the operation O
- 3: Attach the operation with its timestamp to the character node.
- 4: $SV_S[R] \leftarrow SV_S[R] + 1$
- 5: Retracing($Docs, SV_S$)

order of any two character nodes is certain, Retracing($Docs, SV_O$) ensures that the address space (effective character nodes) of the current document's liner structure is the same as the one at the time of the operation's generation. So control algorithms ensure that we can count to the correct effective character node when executing an operation. And by adjusting SV_S , Retracing($Docs, SV_S$) can restore the address space to the state which has included the new operation's execution effect. See also Fig.4. When user U_1 intends to execute operation Insert[“c”, 3] with timestamp $\langle 1, 1, 2 \rangle$, the document state is shown in top-left. First, the control algorithm retraces the document to the view “abd” (top-right) at the time of timestamp $\langle 1, 1, 2 \rangle$ by concealing the effect of operation Delete[“b”, 2], then inserts “c” between “b” and “d”. The State Vector of the local site has been adjusted to $\langle 2, 1, 2 \rangle$ after the execution is finished. And then it retraces to the view “acd” (bottom-left) of new State Vector by restoring the effect of operation Delete[“b”, 2] and adding the execution effect of the new operation Insert[“c”, 3].

3.4 Operation Execution

The focus of this section is on how an operation O is executed:

If O denotes a Delete operation, just find the characters in the address space (count the effective character nodes in the liner structure of $Docs$) to find the position.

If O denotes an Insert operation, find the position in

the address space (count the effective character nodes in the liner structure of $Docs$ to find the range between two adjoining effective character nodes), build a new character node, initialize the mark and Insert it to an accurate position within the range.

Since Retracing($Docs, SV_O$) transforms the address space to the state at the time of the operation's generation, the Delete operation's execution is simple. The process is to find the corresponding character node, attach the operation on the character node and then perform the Retracing procedure to give this node a correct effective/ineffective mark. But the Insert is to search for an Inserting position in the address space, which is corresponding to a range in document between two effective character nodes. There may exist some other ineffective character nodes in this range. We need a Range-Scan process to determine the right-and-left relation between the new Insert and these ineffective character nodes and then find an accurate position. A TOrder function, which realizes a total order relation among Insert operations, has been involved in the follow to realize this Range-Scan process. Since the Delete only revises the mark information and has no effect on the order of character nodes, the focus of following discussion is on Insert operations.

To realize the comparability between two Insert operations, we refer to the technique of maintaining operation transformation order[15]. Given the TOrder relation to represent the right-and-left relation between two inserted characters. Given two character nodes CN_a and CN_b . The attached operations are generated at site a and site b , and are timestamped by SV_a and SV_b . We have $TOrder(CN_a) < TOrder(CN_b)$, iff: (1) $sum(SV_a) < sum(SV_b)$, or (2) $a < b$ when $sum(SV_a) = sum(SV_b)$, where $sum(SV) = \sum_{i=0}^{N-1} SV[i]$. The TOrder relation is a total order which is transitive and any two TOrder are comparable.

The process to determine an accurate position of a new operation O is to scan every Insert in this range from left to right. There are two kinds of Insert operations in this

range. Some of these operations are casually before operation O . We force the new *Insert* O to insert on the left side of these *Insert* operations, so when Range-Scan scans to these operations, it will directly end itself (Range-Scan Function, line 12-17). The other *Insert* operations are concurrent with operation O . We compare them by comparing the $TOrder$ relation (Range-Scan Function, line 4-11). The inserting position is saved in the P . The new operation will be inserted on the left side of P . We can sketch the scanning process as Func.3.

Function 3 Range-Scan(CN_a, CN_b, CN_{new}), scan character nodes between CN_a and CN_b , then return the position of CN_{new}

```

1:  $P \leftarrow Null$  and  $CN_{scan} \leftarrow CN_a.right$ 
2: repeat
3:   Compare  $CN_{new}$  with  $CN_{scan}$ 
4:   if  $CN_{scan} \parallel CN_{new}$  then
5:     if  $TOrder(CN_{new}) < TOrder(CN_{scan})$ 
       and  $P = Null$  then
6:        $P \leftarrow CN_{scan}$ 
7:     end if
8:     if  $TOrder(CN_{new}) > TOrder(CN_{scan})$ 
       and  $CN_{scan} \rightarrow P$  then
9:        $P \leftarrow Null$ 
10:    end if
11:  end if
12:  if  $CN_{scan} \rightarrow CN_{new}$  then
13:    if  $P = Null$  then
14:       $P \leftarrow CN_{scan}$ 
15:    end if
16:    exit the loop
17:  end if
18:   $CN_{scan} \leftarrow CN_{scan}.right$ 
19: until  $CN_{scan} = CN_b$ 
20: if  $P \neq Null$  then
21:   return  $P$ 
22: else
23:   return  $CN_b$ 
24: end if

```

Retracing process locates the *Insert* into a one-and-only range. Range-Scan process determines a one-and-only position for this *Insert* in the range and keeps it from being affected by concurrent operations. Then we explain how algorithm maintains the convergence and intention preservation in detail in the next section.

3.5 Achieve Convergence and Intention Preservation

Suppose that originally, every site has the same initial document state and that all the character node orders at every site are the same. Given n operations to be executed, we shall discuss how to ensure that all the sites still have the same character nodes orders after the operations' execution.

3.5.1 Two operations

Consider the situation where there are two operations O_1 and O_2 . If they are casually ordered, since the execution order is certain, the result will be certain too. If they are concurrent, we prove the following theorem.

THEOREM 1. *If O_1 and O_2 are concurrent and inserted to a same scan-range (i.e. the range between two effective*

characters CN_a and CN_b in the Range-Scan process), the execution effects of two operations have no relation with their execution order.

PROOF. When O_1 and O_2 are executed respectively in this range, there are only three possible cases:

1. O_1, O_2 ends with the same return position.
2. O_1 ends with the return position on the left of O_2 's.
3. O_1 ends with the return position on the right of O_2 's.

We would like to prove that in the three cases above, if we execute one operation after another (i.e. O_1 before O_2 or O_2 before O_1), the results are the same no matter in what sequence they are executed. With out loss of generality, assume $TOrder(O_1) > TOrder(O_2)$.

To simplify description, given the following notations in Fig.5,6.

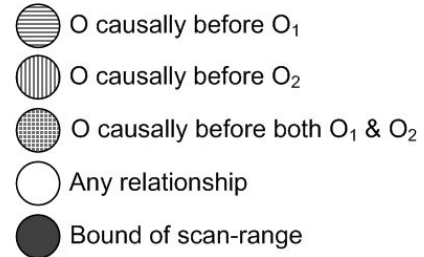


Figure 5: Definition of some special character nodes

$T_{G12} : T_{G12} > TOrder(O_1), T_{G12} > TOrder(O_2)$
 $T_{any} : \text{Any total order}$
 $T_{G1} : T_{G1} > TOrder(O_1)$
 $T_{G2} : T_{G2} > TOrder(O_2)$
 $T_{L1} : T_{L1} < TOrder(O_1)$
 $T_{L2} : T_{L2} < TOrder(O_2)$
 $T_{L1G2} : T_{L1G2} < TOrder(O_1), T_{L1G2} > TOrder(O_2)$

Figure 6: Definition of some special $TOrder$

Case 1: O_1 and O_2 end with the same return position when they are executed respectively.

In this case, the possible situations before the execution can only be Fig.7.

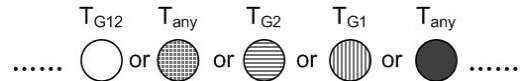


Figure 7: Possible situations before executing O_1 and O_2 respectively in case 1: O_1 and O_2 end with the return position $O_{T_{G12}}$ or $O_{T_{any}}$ or $O_{T_{G2}}$ or $O_{T_{G1}}$ or $O_{T_{any}}$

Then in the situation in Fig.7, if O_1 is executed before O_2 , the result is Fig.8.

If O_2 is executed before O_1 , the same result can be obtained due to the certain $TOrder$ relationship between O_1 and O_2 .

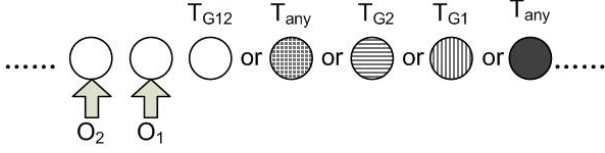


Figure 8: The result of case 1

Case 2: O_1 ends with the return position on the left of O_2 's when they are executed respectively.

In this case, the possible situations before the execution can only be Fig.9.

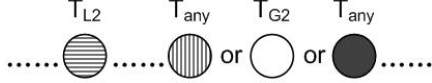


Figure 9: Possible situations before executing O_1 and O_2 respectively in case 2: O_1 ends with the return position O_{TL2} ; O_2 passes through O_{TL2} and ends with the return position O_{Tany} or O_{TG2} or O_{Tany} .

Then in the situation in Fig.9, if O_1 is executed before O_2 , the result is shown in Fig.10-a. O_1 ends with the return position O_{TL2} . During the Range-Scan process of O_2 , when it scans to O_1 , it will record O_1 into P and go on scanning to O_{TL2} . Since O_{TL2} 's $TOrder$ is less than O_2 's and O_{TL2} is causally before O_1 which is saved in P , so P will be reset to $Null$. And then the process ends with the return position O_{Tany} or O_{TG2} or O_{Tany} .

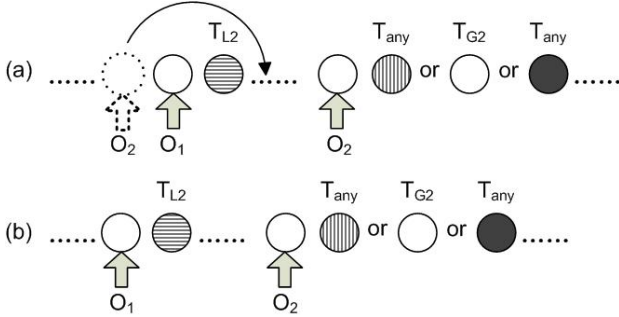


Figure 10: The result of case 2

If O_2 is executed before O_1 , the result is shown in Fig.10-b. O_2 passes through O_{TL2} and ends with the return position O_{Tany} or O_{TG2} or O_{Tany} , and O_1 ends with the return position O_{TL2} as before. We can see that the two results in Fig.10-a and Fig.10-b are the same.

Case 3: O_1 ends with the return position on the right of O_2 's when they are executed respectively.

In this case, the possible situations before the execution can only be Fig.11.

Then in the situation in Fig.11, if O_1 is executed before O_2 , the result is shown in Fig.12. Because O_1 is concurrent with O_{TL1} and O_{TL1G2} , and the $TOrder$ of O_1 is more than that of O_{TL1} and O_{TL1G2} , O_1 will pass through O_{TL1} or O_{TL1G2} and end with the return position O_{TG1} or O_{Tany} or O_{Tany} . When O_2 is being executed, it will end with the return position O_{TL1} or O_{TL1G2} .

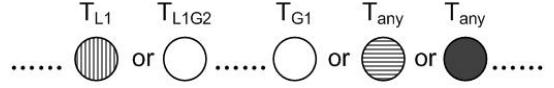


Figure 11: Possible situations before executing O_1 and O_2 respectively in case 3: O_2 ends with the return position O_{TL1} or O_{TL1G2} ; O_1 passes through O_{TL1} or O_{TL1G2} and ends with the return position O_{TG1} or O_{Tany} or O_{Tany} .

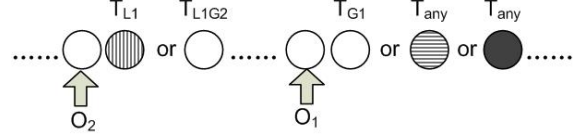


Figure 12: The result of case 3

If O_2 is executed before O_1 , O_2 ends with the return position O_{TL1} or O_{TL1G2} as before. When O_1 is executed, since it is concurrent with O_2 and its $TOrder$ is more than that of O_2 , it will pass through O_2 as well as O_{TL1} or O_{TL1G2} and end with return position O_{TG1} or O_{Tany} or O_{Tany} . Thus the same result is obtained.

All the three cases show that no matter in what sequence O_1 and O_2 are executed the results are always the same, i.e. the execution effects of the two operations have no relation with their execution order. $\square$

THEOREM 2. If O_1 and O_2 are concurrent, the execution effects of two operations have no relation with their execution order.

PROOF. Consider the scan-range of the two operations.

1. If the scan-ranges of the two operations have no shared part, it is obvious that the two operations do not affect each other due to the retracing process.
2. If the scan-ranges of the two operations have one shared part. As shown in Fig.13, $\overrightarrow{AC}$ denotes O_1 's scan-range, $\overrightarrow{BD}$ denotes O_2 's scan-range. The relative position (left or right) of C and D (two right bounds of the scan-ranges) is ignored in our following proof since it has no impact on the proof of Theorem 1. Then consider two cases:

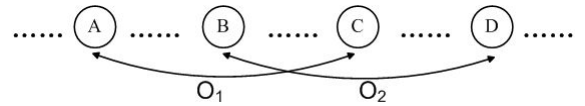


Figure 13: Scan-Range of two operations

- (a) $A \neq B$. Without loss of generality, assume $A < B$. If O_1 ends with a return position in $\overrightarrow{AB}$, it is obvious that the operations do not affect each other. If O_1 ends with a return position in $\overrightarrow{BC}$, we can merge this case into the second case $A = B$.
- (b) $A = B$. Suppose the two operations are done respectively (i.e. do O_1 , undo O_1 and then do O_2), we have already proved that the execution effects

of two operations have no relation with their execution order in a same range and the relative position of the right bounds has no impact on this result.

Therefore, the execution effects of two operations have no relation with their execution order no matter what scan-ranges they have. $\square$

3.5.2 N operations

THEOREM 3. *Orders of character nodes at every site are still consistent when n operations have been executed.*

PROOF. First, when all the document orders at every site are consistent, executing two operations under the execution condition, a consistent order is certain to be obtained. Then suppose the orders of character nodes at every site are consistent after we have executed $n - 1$ operations in the order which meets the execution condition. What we want to prove is that the orders of character nodes at every site are still consistent when n operations have been executed.

Suppose executing n operations under the execution condition at two sites, whose order can be expressed as $O_1O_2O_3 \dots O_n$ and $O_{k_1}O_{k_2}O_{k_3} \dots O_{k_n}$. Without loss of generality, suppose $O_{k_i} = O_1$. Since $O_1O_2O_3 \dots O_n$ meets the execution condition, O_{k_i} and $O_{k_1}O_{k_2}O_{k_3} \dots O_{k_{i-1}}$ are concurrent. The change of any two concurrent operations' execution order does not affect the result of the document, so we can switch O_{k_i} with $O_{k_{i-1}}, O_{k_{i-2}}, O_{k_{i-3}}, \dots, O_{k_2}$ and O_{k_1} in turn and obtain $O_{k_i}O_{k_1}O_{k_2} \dots O_{k_{i-1}}O_{k_{i+1}} \dots O_{k_n}$. Since $O_{k_i} = O_1$ and after the adjustment the first operation of every sequence is O_1 , we can execute the first operation of every sequence first. Here, all the sites' orders are still consistent and the operation sequences to be executed are changed to $O_2O_3 \dots O_n$ and $O_{k_2}O_{k_3} \dots O_{k_{i-1}}O_{k_{i+1}} \dots O_{k_n}$ which still meet the execution condition. According to the assumption, the orders of character nodes at every site are consistent after we have executed $n - 1$ operations in the order which meets the execution condition, so the orders of document's character nodes are still consistent after these two sequences have been executed. So all the orders of character node at the sites' are consistent after n operations have been executed in the order which meets the execution condition. $\square$

3.5.3 Summary

We have proved that after executing several operations in any order which meets the execution condition, the orders of character nodes at every site will keep consistent. It is to say that the order in which two operations are executed at one site for the first time has been kept at any other sites. In addition, the key to the correctness of a *Delete* operation is that it can find the correct effective character node and the consistency of orders of character node ensures it to count to the correct effective character node. Both of the orders of character nodes and effective/ineffective marks of every character node at all sites are kept consistent, so the documents are convergent. Since a *Insert* operation can find the correct scan-range and a *Delete* operation can find its corresponding character node at any sites and they do not interfere with other operations, those satisfy operations' definition in Section 2. However, there are still some arguments for intention preservation, we plan to present more about intention's formal definition and the complete proof in the next future.

3.6 Some Existing Puzzles

The dOPT puzzle described in C.A.Ellis et al.[2] supposes that several *Insert* operations targeting the same position satisfy certain causal or concurrent relationships. Judging their order by simple priority cannot ensure convergence. In the dOPT puzzle, different operations execution orders can be reduced to the two typical cases shown in Fig.10. In Fig.10, O_1 is causally after $O_{T_{L_2}}$ and O_2 are concurrent with O_1 and $O_{T_{L_2}}$. It is shown that we can always obtain the same result no matter in what sequence the operations are executed. It is proved in Section 3.5.2 that the convergence can also be ensured in some more complex situations.

The false-tie puzzle in C.Sun et al.[15], which is also called ERV puzzle in D.Li et al.[6], supposed that two concurrent *Insert* operations may lose their relative position (left or right) due to the *Delete* between them and obtain the opposite result. In the Mark & Retrace method, the scan-ranges of two *Insert* operations with the relative position (left or right) have no shared part. Since a concurrent *Delete* has no impact on the result of the retracing process, the range-scan processes of these two *Insert* operations are performed in two independent scan-ranges and do not affect each other. So the results of every site are convergent.

4. EFFICIENCY ANALYSIS

In Control-algorithm, we can see that the cost of algorithm is mainly spent on two aspects. One is the execution of Retracing and the other is the execution of *Insert* or *Delete*. Attaching operations on corresponding nodes and modifying the state of the local site can be finished in time $O(1)$. Discuss these two aspects respectively. Since the execution efficiency of Retracing is relevant to the operations' organize structure, we first introduce the optimization of operation execution efficiency.

4.1 The Optimization of Operation Execution Efficiency

The first step of operation's execution is to count to the appointed nodes. Then the *Insert* chooses an inserting position between two nodes during the Rang-Scan proc while the *Delete* only revises that node's effective/ineffective mark. The efficiency of mark revision is obviously $O(1)$ and the time complexity of Rang-Scan is $O(m)$ in which m represents the number of the ineffective character nodes between the two effective character nodes. When the document is of great scope, comparing with the great number of ineffective character nodes produced by *Delete*, the concurrent operations which are marked ineffective can be neglected. So m is approximately equal to the using rate of *Delete* and *Insert*.

When a document is organized by liner structure, the process of counting is sequential access to linear structure. Its time complexity is linear which is expressed as $O(n)$ in which n is the number of character nodes in the document. But when the document is of great scope, the great number of ineffective nodes makes the execution time of algorithm dissatisfactory. We hope to further optimize the operation execution efficiency.

We reorganize the character nodes to a *Red-Black Tree*[1] and ensure the result of pre-order traversal of the Black-Red tree is the original linear order. It means that every node's left sub-tree is the set of the nodes which are on the left of itself in the original linear structure. As the figure

below, the *Insert* and *Delete* operations in the *Red-Black Tree* can keep this property easily. Since the internal rotation rules of *Red-Black Tree* do not affect the result of pre-order traversal, as long as the counting process can find the correct position, the correctness of algorithm will not be affected if we change the sequential access from left to right to the pre-order traversal of the tree.

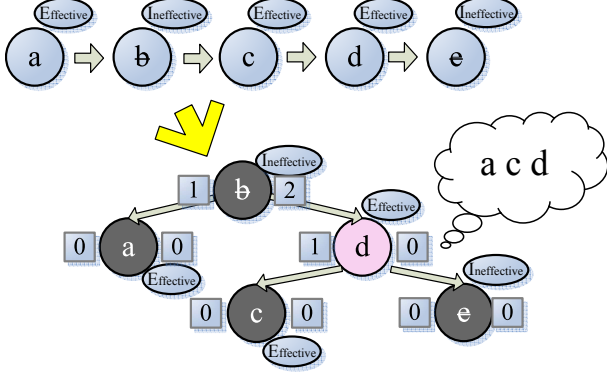


Figure 14: Adjusting the liner structure to a *Red-Black tree*

Then we extend the data structure of original *Red-Black Tree*. Every node of the tree saves two additional data —the two numbers of the effective character nodes of left sub-tree and right sub-tree. When the effective/ineffective mark of every node has been changed or there is a effective character node being Inserted or deleted, only the data of ancestor of this node has to be changed. Suppose a *Red-Black Tree* has n nodes (n is the number of the document's character nodes). The time complexity is $O(\log n)$ when changing the mark of one node because of the tree is approximately balanced. In one *Red-Black Tree*, one *Insert* or *Delete* operation can result in at most two or three rotations and every rotation only involves two nodes. We can adjust the data of this node (the number of effective character nodes of this node's left and right sub-tree) when executing rotation operations. The time complexity of changing the two nodes' mark is $O(1)$ and the time complexity of ancestor's adjustment is $O(\log n)$, so the time complexity of *Insert* or *Delete* in one *Red-Black Tree* is still kept $O(\log n)$. The reader can derive the analysis and the proof of the correctness of expanded *Red-Black Tree* in [1].

Then when we need to count the P th effective character node in pre-order traversal, we can search it recursively as Func.4.

This recursively searching process is the extension of the general traversal process of binary search tree, whose time complexity is still $O(\log n)$.

So in the new document structure, the optimization of an *Insert* operation's time complexity is $O(\log n + m)$ and the *Delete* operation's is $O(\log n)$. Since m is the using rate of *Delete* and *Insert*, we can use ordinary potential method to make the further analysis: since the *Delete* is executed m times while the *Insert* is executed once, we can amortize the cost $O(m)$ of Range-Scan within an *Insert* to m corresponding *Delete* operations. And then the amortized costs of *Insert* and *Delete* are both $O(\log n)$.

Function 4 Count($P, node$), return the P th effective character node

```

1: if  $node.NumofLeft = P-1$  and  $node.effective = true$ 
   then
2:   return node
3: end if
4: if  $node.NumofLeft > P-1$  then
5:   return Count( $P, node.left$ )
6: else
7:   if  $node.effective = true$  then
8:     return Count( $P - node.NumofLeft - 1, node.right$ )
9:   else
10:    return Count( $P - node.NumofLeft, node.right$ )
11:   end if
12: end if

```

4.2 Efficiency Optimization of Retracing

The Retracing procedure is to retrace to the state according to the given timestamp SV . The effective/ineffective mark is only determined by the relations between SV and timestamp of every operation attached on the node. When the timestamp of every operation attached on the node are kept causal before SV , the node's effective/ineffective mark is certain. In other words, for every operation to be executed, only when the operation attached on a certain node is concurrent with it, this node's effective/ineffective mark will be changed.

On the one hand, suppose an operation O has been executed at every site, and another operation O' generates after O . There must be $O \rightarrow O'$ and there is no need to consider operation O when we executing Retracing by the timestamp of operation O' . On the other hand, every operation is sent to other sites together with SV of current site. This SV represents the documents state at the time of operation's generation and through it other sites will know what operations have been executed at the sending site. We only keep the $OList$ containing the operations which haven't been executed at some sites. During Retracing process, we only take into account of the character nodes relevant to these operations into consideration and as long as we have ensured that one operation has been executed at every site, we can delete it from the $OList$.

Then we can use the same method as State Vector Table(SVT) in Garbage Collection Scheme in [15] to determine the members of $OList$. Maintain a SVT at every site and suppose the state vector table of the k th site is SVT_k . Initially, $SVT_k[i][j] = 0$ for all $i, j \in 0, \dots, N-1$. After executing an operation O from a remote site r , timestamped by SV_O , updated the SVT_k : $SVT_k[r][i] = SV_O[i]$, for all $i \in 0, \dots, N-1$. Let MSV_k be the minimum state vector at site k : $MSV_k[i] = \min(SVT_k[0][i], \dots, SVT_k[N-1][i])$, for all $i \in 0, \dots, N-1$. MSV_k is updated after the new operation's execution at site k . Then check every operation $OList-i$ in $OList$. Suppose $OList-i$ is generated at site r and timestamped by $SV_{OList-i}$. When $SV_{OList-i}[r] \leq MSV_k[r]$, it is obvious that $OList-i$ has been executed at all sites. So we can delete it from the list.

Suppose the length of $OList$ is h which means we need to check and change at most h character nodes' effective/ineffective mark during Retracting process. Under the new document structure, suppose the number of operations attached on a

character node is d and then the cost of checking the node's effective/ineffective mark is $O(d)$ and the cost of adjusting its ancestor's data (the number of effective character nodes of node's left and right sub-tree) is $O(\log n)$. So the time complexity of Retracing can be expressed as $O(d \cdot h + h \cdot \log n)$. Generally, the number of the operations attached on a node is smaller than ten, so we can consider d as a constant. During the operation execution process, when the operation is executed and fed back in good time, the length of *OList* will not be increased. Obviously, we can also consider h as a constant.

Before the execution of Control-Algorithm process, the state vector of site S is SV . Control-Algorithm adjusts this state vector to SV' by $SV'_S[R] \leftarrow SV_S[R] + 1$ and only the R th element has been changed. So after the second Retracing process in Control-Algorithm is finished, only the targeting character node of the new operation has changed. We record all the changed nodes in the Retracing(Doc_S , SV_O) first. When it is the time to execute Retracing(Doc_S , SV_O), we restore the changed nodes instead of the Retracing process. After then, we only need to deal with the targeting character node of the new operation. The time complexity of the restore process can be expressed as $O(h + d)$.

4.3 General Efficiency Analysis

Colligate the execution efficiency of operation and process of Retracing, we have the actual execution cost. The cost of *Insert* is $O(h \cdot \log n + \log n + d \cdot h + h + d + m)$ and of *Delete* is $O(h \cdot \log n + \log n + d \cdot h + h + d)$. In the situation where there are great number of operations, consider h (the *OList*'s length) and d (the number of operations attached on every node) as two constants, we have the amortized cost of an operation $O(\log n)$, in which n is the number of character nodes in the document. And we plan to give a further comprehensive comparison with OT technique in the future work.

5. UNDO SUPPORTED

In Mark & Retrace method, the expression of *Undo* is like that of *Insert* and *Delete*. It includes the state vector of its generation site and the operation that is being undone. We can identify the corresponding operation of the *Undo* by the number of generation sites and the timestamp attached [19, 20]. When the *Undo* operation is causally-ready, it will be executed in the control algorithms. Given an *Undo* operation O_2 , Undo[O_1], in which O_1 's target position is P_1 and timestamped by SV_{O_1} . O_1 and O_2 should be attached on the same character node. The execution process can be performed as follow. Rretrace the address space to the state at the time SV_{O_1} , count to the P_1 th effective character node and attach O_2 on it. Then the second Retracing process takes this new *Undo* into consideration and gives this node the correct effective/ineffective mark.

During the Retracing process, the only adjustment produced by the introduction of *Undo* operation is that it takes the effect of *Undo* into consideration when deciding the effective/ineffective mark of a character node. Since there may attached several concurrent *Delete* operations or *Undo* operations on one character node, it may cause the intention violation [19, 20]. We have discussed the cause of intention violation and how to judge and deal with it in [19, 20], but we have not discussed the display strategy in detail. Actually, the method will be very simply if we want to realize Multi-version/Single-display [12, 13, 16]. The advantage

of Multi-version/Single-display has been also emphasized in CoWord/CoPowerPoint [12, 13, 16].

In order to realize the Multi-version/Single-display strategy of intention violation by *Undo*, we append a *Delete* counter to every character node, whose initial value is zero. Every *Delete* and every *Undo* toward *Insert* will plus 1 to this counter while every *Undo* toward *Delete* will minus 1 to this counter. The Retracing process only considers the effects of other valid operations (the timestamp is causally before the given one) on the counter. Consider a character node, when the *Insert* is valid and the counter's value is 0, this character is marked effective, otherwise it will be marked ineffective. When there are several *Delete* operations in the algorithm, the character node is effective only when all the *Delete* operations are undone, otherwise it will keep ineffective. When there exists intention violation, the algorithm maintains the intention of users who want character node to be ineffective, namely Multi-version/Single-display strategy [16].

During the revised Retracing process, the set of valid operations in a given SV is certain so the result of the counter is also certain and the effective/ineffective mark as well. Since the *Undo* does not affect the order of character nodes, the CCI model is also maintained. Compared with OT, Mark & Retrace with *Undo* operation can deal with current *undo* puzzles well and it does not need any other adjustments [10, 11]. For space reason, please refer to [19, 20] for the analysis of how to deal with undo puzzles and the cause and judgments of intention violation. The execution efficiency of undo operation approximates that of *Delete*. The time complexity and its amortized cost are both kept as $O(\log n)$.

6. RELATED WORKS

The initial concept of Address Space Transformation was reported early in [4, 18] but was not able to resolve the dOPT puzzle. The algorithms in [4, 18] are localized in the relation between the new operation and the executed ones at current site but has not fully considered the confusion of concurrent operations. When all the operations in dOPT puzzle are executed between two existing character nodes (for example, executing dOPT puzzle in an initially vacant document), the operation order will not be affected by the retracing process. So it needs additional solutions when there are several concurrent operations targeting one position, but [4, 18] lack such support to this situation.

Algorithm dOPT, adOPTed and GOTO are based on TP2, [2, 7, 14, 15], but cannot fully satisfy TP2. NICE reported notification server [8] and SOCT3/4 used the sequence method [17], but neither of them is able to resolve the false-tie (ERV) puzzle. GOT maintains the convergence by the method of a global total order of execution. But Li et al.[6] pointed out that the original GOT will fail in the case of complicated false-tie (ERV) scenarios but the details have not been published. Li et al.[6] also pointed out that although the solution of false-tie (ERV) has been realized in [12], it is still not published. Imine et al.[5] tried to support TP2 by expanding IT function but fails in solving dOPT puzzle. Li et al.[6] reported a solution and also gave the proof. We have explained that our Mark & Retrace method can satisfy the CCI model and deal with dOPT puzzle and false-tie(ERV) puzzle.

We discuss the support to *Undo* based on the current work, finally. Among the algorithms above, adOPTed sup-

ports the sequential *Undo* by maintaining a multiple dimensions data structure [7]. REDUCE supports undo any operation at any time by inversing the Undo operation [10, 11]. But when facing Undo puzzles, it needs to maintain the operations' relationship additionally, which adds to the time complexity of algorithm (nonlinear time complexity). Mark & Retrace method also can support any *Undo* and can resolve the *Undo* puzzles in [10] more easily. None of the current methods has considered the intention violation in *Undo* [19, 20]. Although REDUCE can maintain convergence in the intention violation and obtain the effect of Multi-version/Single-display strategy, it lacks related examining method and enough hints to users. In this paper, we report a new solution to Multi-version/Single-display strategy and plan to discuss the Multi-version/ Multi-display strategy and its related examining and hinting method in the further work. More information can be derived in [19, 20].

7. CONCLUSIONS

In this paper we present a strategy which can maintain the CCI model and is different from OT. It does not ensure the operation's execution correctness by modifying the operation's address but by transforming the address space of the document. Since the Mark & Retrace processes do not affect the relative position (left or right) between characters, it is easier to maintain operations' order. It can not only achieve all the consistency criteria in [6, 15], but can support any Undo as well.

For space reason, we have not given more examples to describe the process to resolve the Undo puzzles using Mark & Retrace. More information can be derived in [19, 20]. We plan to discuss more about the Multi-version/Multi-display strategy in Undo intention violation and the support to update operation and tree based structures of Mark & Retrace Algorithm.

8. ACKNOWLEDGMENTS

The authors wish to thank Yan Yu and David Sun for their help in improving the final presentation of this article and Qin Zhang for his assistance in the proofs. The authors are also very grateful to anonymous referees for their detailed and insightful comments and suggestions.

9. REFERENCES

- [1] T. H. Cormen, C. E. Leiserson, and R. L. Rivest. *Introduction to Algorithms*. MIT Press, Boston, Sept 2001.
- [2] C. A. Ellis and S. J. Gibbs. Concurrency control in groupware systems. In *Proceedings of the ACM Conference on the Management of Data 1989*, pages 399–407, Portland Oregon, May 1989. ACM.
- [3] N. Gu, J. Xu, X. Wu, J. Yang, and W. Ye. Ontology based semantic conflicts resolution in collaborative editing of design documents. *International Journal of Advanced Engineering Informatics (IJAEI)*, Jan 2005. Accepted.
- [4] H. He, Q. Wu, and L. Luo. Document marking scheme for preserving intention of operation in cooperative editing system. *Journal of Software*, 10(2):160–164, Feb 1999.
- [5] A. Imine, P. Molli, G. Oster, and M. Rusinowitch. Proving correctness of transformation functions in real-time groupware. In *European CSCW'03 Proceedings*, Sept 2003.
- [6] D. Li and R. Li. Preserving operation effects relation in group editors. In *Proceedings of ACM Conference on CSCW 2004*, pages 437–446, Chicago, USA, Nov 2004. ACM.
- [7] M. Ressel, D. Nitsche-Ruhland, and R. Gunzenbauser. An integrating, transformation-oriented approach to concurrency control and undo in group editors. In *Proceedings of ACM Conference on CSCW 1996*, pages 288–297, Nov 1996.
- [8] H. Shen and C. Sun. Flexible notification for collaborative systems. In *Proceedings of ACM Conference on CSCW 2002*, pages 77–86, Chicago, USA, Nov 2002.
- [9] M. Suleiman, M. Cart, and J. Ferrie. Concurrent operations in a distributed and mobile collaborative environment. In *Proceedings of the IEEE Fourteenth International Conference on Data Engineering*, pages 36–45, Orlando, USA, Feb 1998.
- [10] C. Sun. Undo any operation at any time in group editors. In *Proceedings of ACM Conference on CSCW 2000*, pages 191–200, Philadelphia, USA, Dec 2000.
- [11] C. Sun. Undo as concurrent inverse in group editors. *ACM Transactions on Computer-Human Interaction*, 9(4):309–361, Dec 2002.
- [12] C. Sun. The cword and copowerpoint project. <http://reduce.qpsf.edu.au/coword/>, 2004.
- [13] C. Sun and D. Chen. Consistency maintenance in real-time collaborative graphics editing systems. *ACM Transactions on Computer-Human Interaction*, 9(1):1–41, Mar 2002.
- [14] C. Sun and C. A. Ellis. Operation transformation in real-time group editor: issuer, algorithm, and achievement. In *Proceedings of ACM Conference on CSCW 1998*, pages 59–68, Seattle, USA, Nov 1998.
- [15] C. Sun, X. Jia, Y. Zhang, Y. Yang, and D. Chen. Achieving convergence, causality-preservation, and intention-preservation in real-time cooperative editing systems. *ACM Transactions on Computer-Human Interaction*, 5(1):63–108, Mar 1998.
- [16] D. Sun, S. Xia, C. Sun, and D. Chen. Operational transformation for collaborative word processing. In *Proceedings of ACM Conference on CSCW 2004*, pages 437–446, Chicago, USA, Nov 2004.
- [17] N. Vidot, M. Cart, J. Ferrie, and M. Suleiman. Copies convergence in a distributed realtime collaborative environment. In *Proceedings of ACM Conference on CSCW 2000*, pages 171–180, Philadelphia, USA, Dec 2000.
- [18] X. Wu and N. Gu. A concurrency control method based on document marking. *Journal of Computer Research and Development*, 39(12):1662–1667, 2002.
- [19] J. Yang, N. Gu, and X. Wu. A document mark based on method supporting group undo. In *ACM CSCW'2004 Workshop on Collaborative Editing Systems*, Chicago, USA, Nov 2004.
- [20] J. Yang, Q. Zhang, N. Gu, G. Yang, and Z. Liu. The multi-version and single-display strategy in undo scheme. In *The 5th International Conference on Computer and Information Technology*, Shanghai, China, Sept 2005.